МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федерального государственного бюджетного образовательного учреждения высшего образования

«РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ Г.В.ПЛЕХАНОВА»

Техникум Пермского института (филиала)

Практическая работа №2

по дисциплине «Поддержка и тестирование программных модулей»

Руководитель: Д.Б. Берестов /И.О. Фамилия/

“__22__” _марта_2026г

Исполнитель: И.А. Вогусова /И.О. Фамилия/

“__22__” _марта_ 2026 г.

Пермь 2026 г

Оглавление

1.Введение: 3

2.Постановка задачи: 3

3.Выбор средств реализации 3

4. Разработка библиотеки классов 4

4.1 Создание проекта библиотеки классов 4

4.2 Создание перечисления Operation 5

4.3 Создание класса CalculatorEngine 5

5. Разработка WPF-приложения 6

5.1 Создание WPF проекта 6

5.2 Реализация MVVM паттерна 7

5.2.1 Класс RelayCommand 7

5.2.2 Класс MainViewModel 8

5.3 Создание интерфейса (XAML) 10

5.4 Подключение библиотеки логики 11

6. Разработка модульных тестов 11

6.1 Создание тестового проекта 11

6.2 Написание тестов 12

6.3 Запуск тестов 13

7. Сборка и тестирование 14

7.1 Сборка решения 14

7.2 Проверка работы калькулятора 14

Тема: Разработка WPF-приложения «Калькулятор» с использованием библиотеки классов и модульных тестов

1.Введение:

Целью данной практической работы является разработка приложения «Калькулятор» с графическим интерфейсом на платформе WPF (Windows Presentation Foundation). Приложение должно поддерживать базовые арифметические операции: сложение (+), вычитание (-), умножение (×), деление (÷) и возведение в степень (^). В соответствии с заданием, основная логика вынесена в отдельную библиотеку классов, а для её проверки написаны модульные тесты.

2.Постановка задачи:

В ходе выполнения работы необходимо решить следующие задачи:

1. Создать решение в Visual Studio, содержащее три проекта:

   · Библиотека классов (логика вычислений);

   · WPF-приложение (пользовательский интерфейс);

   · Проект модульных тестов.

2. Реализовать логику калькулятора в отдельной библиотеке классов.

3. Разработать интерфейс калькулятора с использованием технологии WPF.

4. Написать модульные тесты для проверки корректности работы математических операций.

5. Опубликовать исходный код проекта на GitHub.

6. Подготовить сопроводительную записку с описанием проделанной работы.

3.Выбор средств реализации

Для выполнения работы были выбраны следующие инструменты:

Компонент: Выбранное средство

Среда разработки: Microsoft Visual Studio 2022

Платформа: .NET Framework 4.8

Технология: GUI Windows Presentation Foundation (WPF)

Фреймворк тестирования: MSTest

Система контроля версий: Git

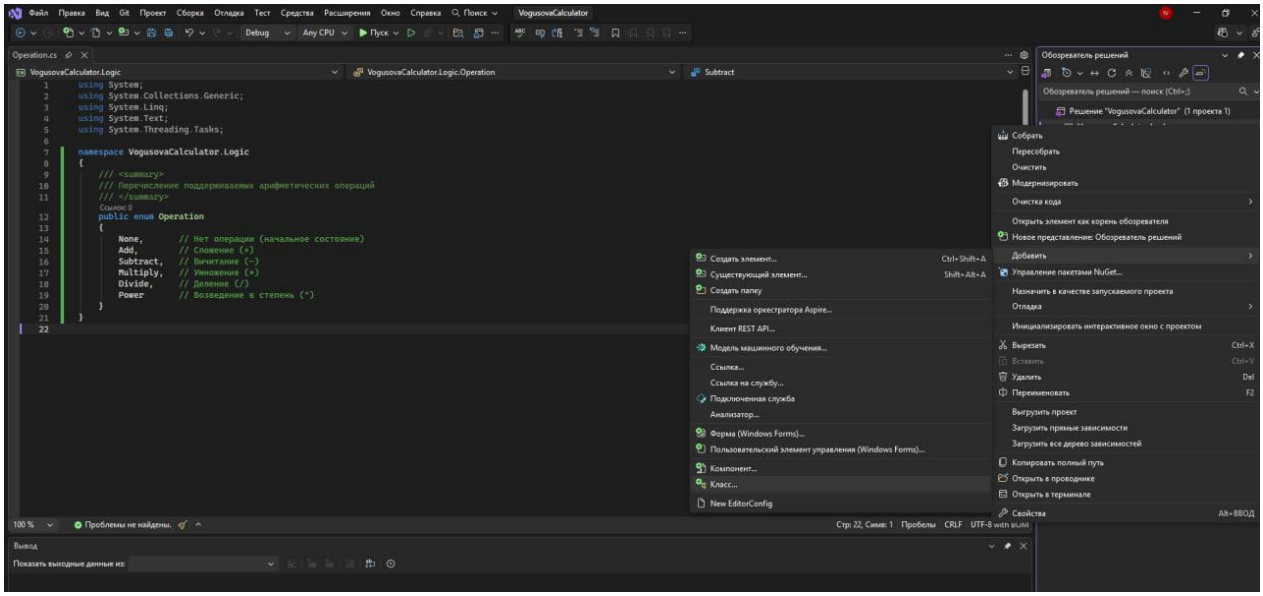
Хостинг кода: GitHub

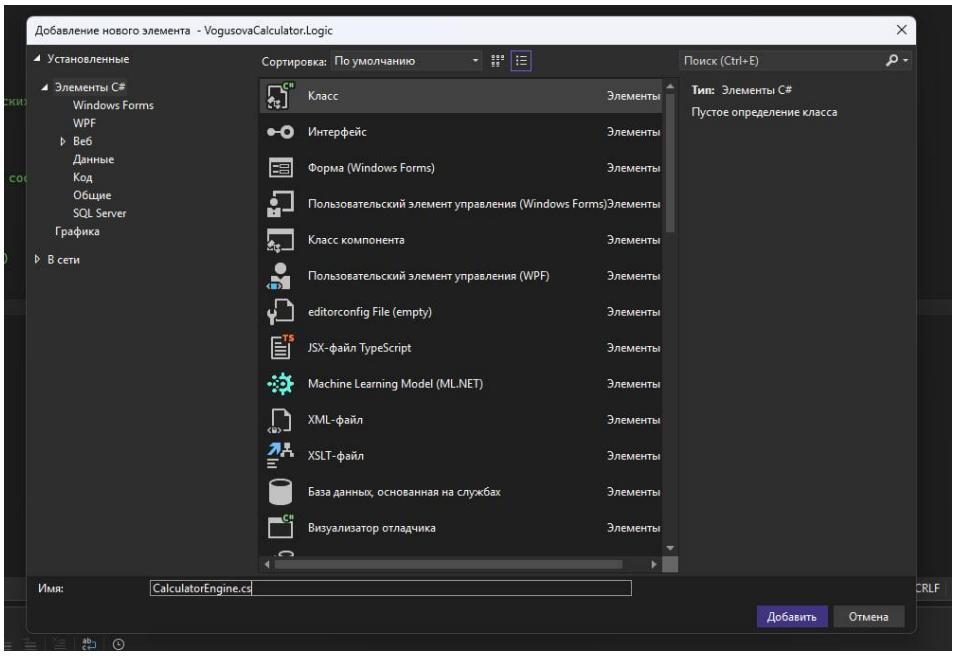
Выбор .NET Framework 4.8 обусловлен необходимостью совместимости с WPF и стабильностью платформы. MSTest выбран как стандартный фреймворк тестирования для C# проектов.

4. Разработка библиотеки классов

4.1 Создание проекта библиотеки классов

Первым шагом было создание решения и проекта библиотеки классов. В Visual Studio был выбран шаблон «Библиотека классов (.NET Framework)» с целевой платформой .NET Framework 4.8. Проект получил имя VogusovaCalculator.Logic.





4.2 Создание перечисления Operation

Для удобства работы с операциями было создано перечисление Operation, определяющее возможные типы вычислений:

csharp

```
public enum Operation
{
    None,       // Нет операции
    Add,        // Сложение (+)
    Subtract,   // Вычитание (-)
    Multiply,   // Умножение (*)
    Divide,     // Деление (/)
    Power       // Возведение в степень (^)
}
```

4.3 Создание класса CalculatorEngine

Основная логика калькулятора реализована в классе CalculatorEngine, который содержит методы для каждой арифметической операции:

csharp

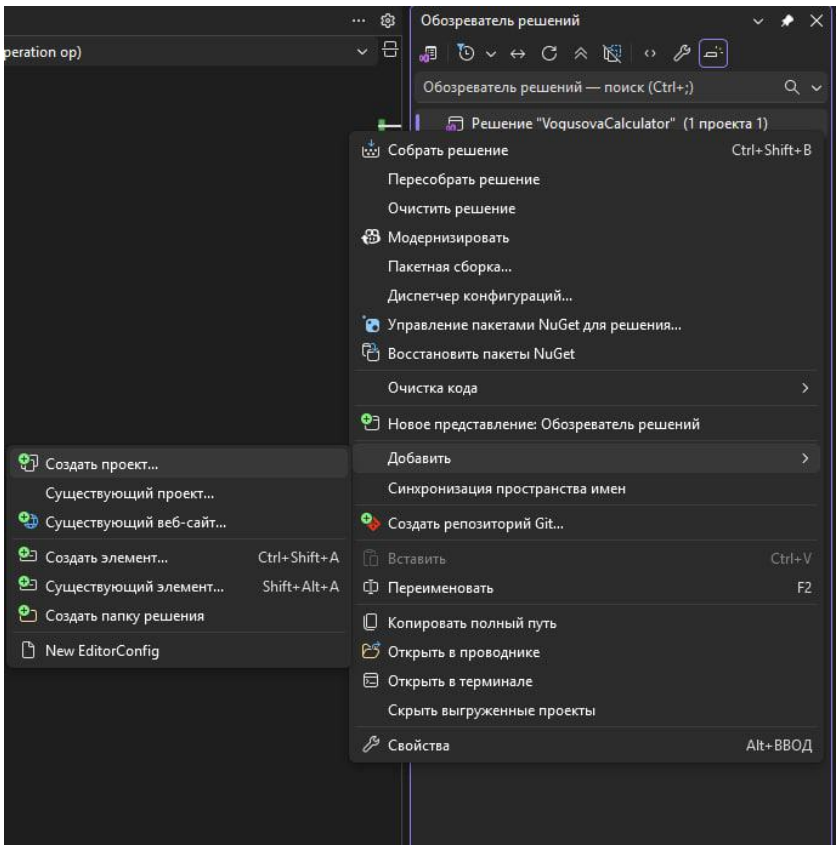
```
public class CalculatorEngine
{
    public double Add(double a, double b) => a + b;
    public double Subtract(double a, double b) => a - b;
    public double Multiply(double a, double b) => a * b;

    public double Divide(double a, double b)
    {
        if (Math.Abs(b) < 1e-10)
            throw new DivideByZeroException("Деление на ноль невозможно.");
        return a / b;
    }
```

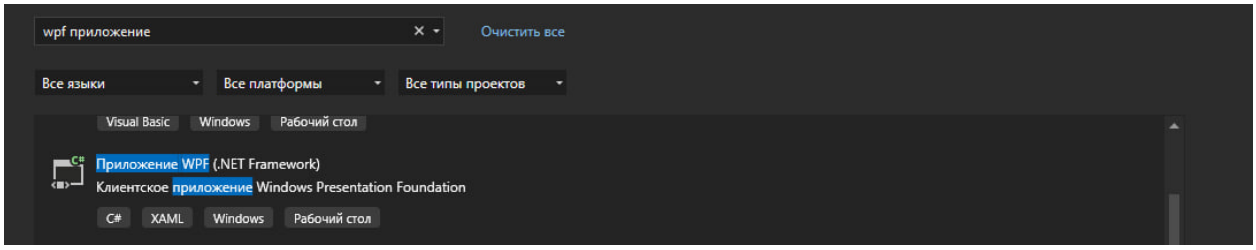
```
    public double Power(double a, double b) => Math.Pow(a, b);


    public double Calculate(double a, double b, Operation op)
    {
        switch (op)
        {
            case Operation.Add: return Add(a, b);
            case Operation.Subtract: return Subtract(a, b);
            case Operation.Multiply: return Multiply(a, b);
            case Operation.Divide: return Divide(a, b);
            case Operation.Power: return Power(a, b);
            default: throw new InvalidOperationException($"Операция '{op}' не поддерживается.");
        }
    }
}
```

5. Разработка WPF-приложения

5.1 Создание WPF проекта

Для реализации пользовательского интерфейса был создан проект VogusovaCalculator.UI с использованием шаблона «WPF App (.NET Framework)».





5.2 Реализация MVVM паттерна

Для разделения логики и представления был использован паттерн MVVM (Model-View-ViewModel). В проекте созданы папки Commands и ViewModel

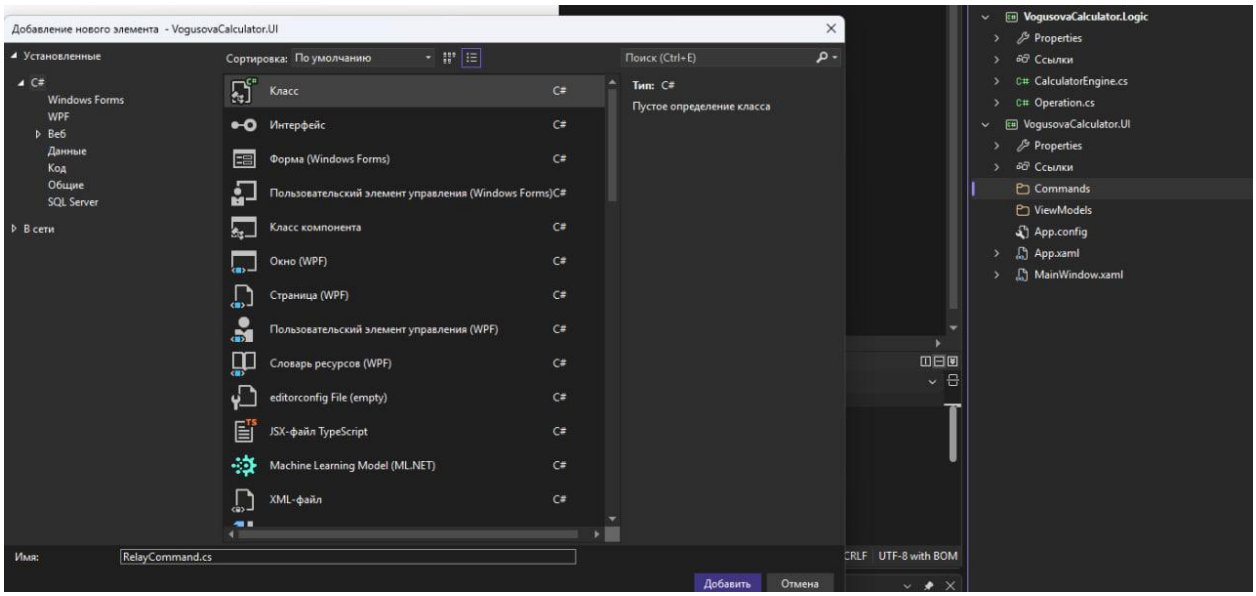
5.2.1 Класс RelayCommand

Класс RelayCommand реализует интерфейс ICommand и позволяет привязывать команды к кнопкам в XAML:

csharp

```
public class RelayCommand : ICommand
{
    private readonly Action<object> _execute;
    private readonly Predicate<object> _canExecute;
```

```
    public RelayCommand(Action<object> execute, Predicate<object> canExecute = null)
    {
        _execute = execute ?? throw new ArgumentNullException(nameof(execute));
        _canExecute = canExecute;
    }

    public bool CanExecute(object parameter) => _canExecute == null || _canExecute(parameter);
    public void Execute(object parameter) => _execute(parameter);

    public event EventHandler CanExecuteChanged
    {
        add { CommandManager.RequerySuggested += value; }
        remove { CommandManager.RequerySuggested -= value; }
    }
}
```

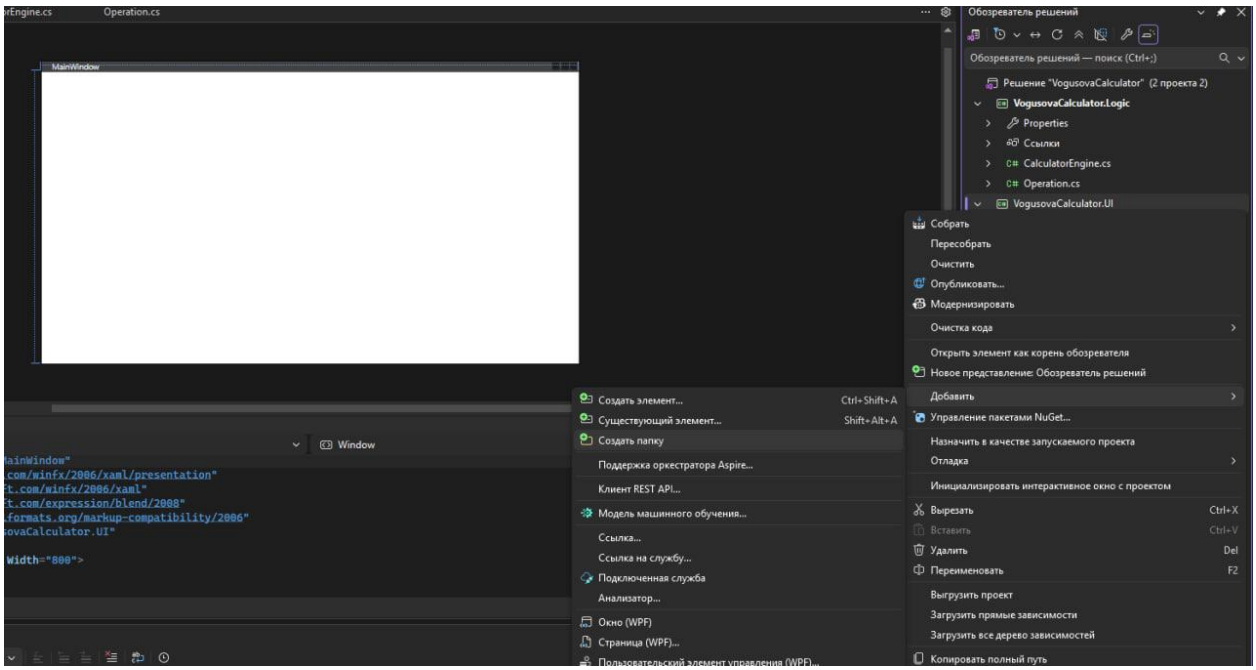


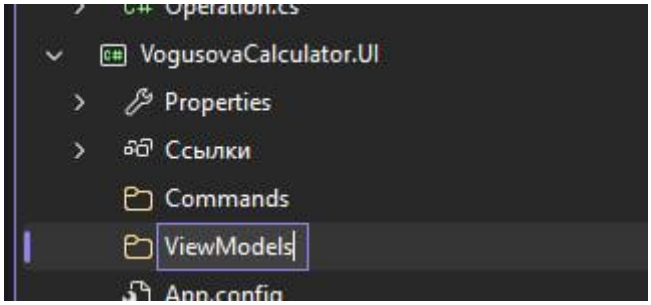
5.2.2 Класс MainViewModel

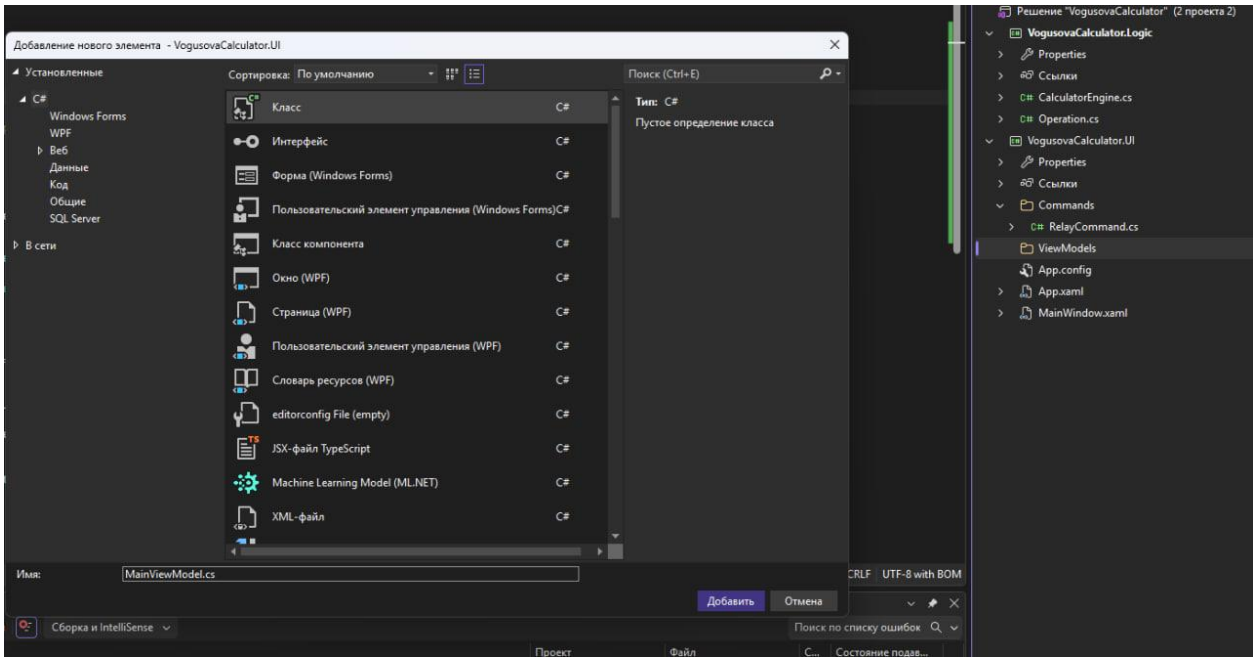
Класс MainViewModel содержит всю логику взаимодействия с интерфейсом: обработку нажатия цифр, операций, вычисление результата и ведение истории.

csharp

```
public class MainViewModel : INotifyPropertyChanged
{
    private readonly CalculatorEngine _engine;
    private string _input;
    private double _currentValue;
    private Operation _currentOperation;
    private bool _isNewInput;

    public MainViewModel()
    {
        _engine = new CalculatorEngine();
        _input = "0";
        History = new ObservableCollection<string>();

        DigitCommand = new RelayCommand(Digit);
        OperationCommand = new RelayCommand(SetOperation);
        EqualsCommand = new RelayCommand(o => Calculate());
        ClearCommand = new RelayCommand(o => Clear());
    }

    // ... остальной код
}
```







5.3 Создание интерфейса (XAML)

Интерфейс калькулятора содержит поле для ввода чисел, сетку кнопок (5×4) и панель истории вычислений.

Основные элементы интерфейса:

· Поле ввода с крупным шрифтом (32pt)

· Кнопки цифр (0-9)

· Кнопки операций (+, -, ×, ÷, ^)

· Кнопка «C» для очистки (красная)

· Кнопка «=» для вычисления (зелёная)

· Кнопки скобок (и)

· Панель истории вычислений

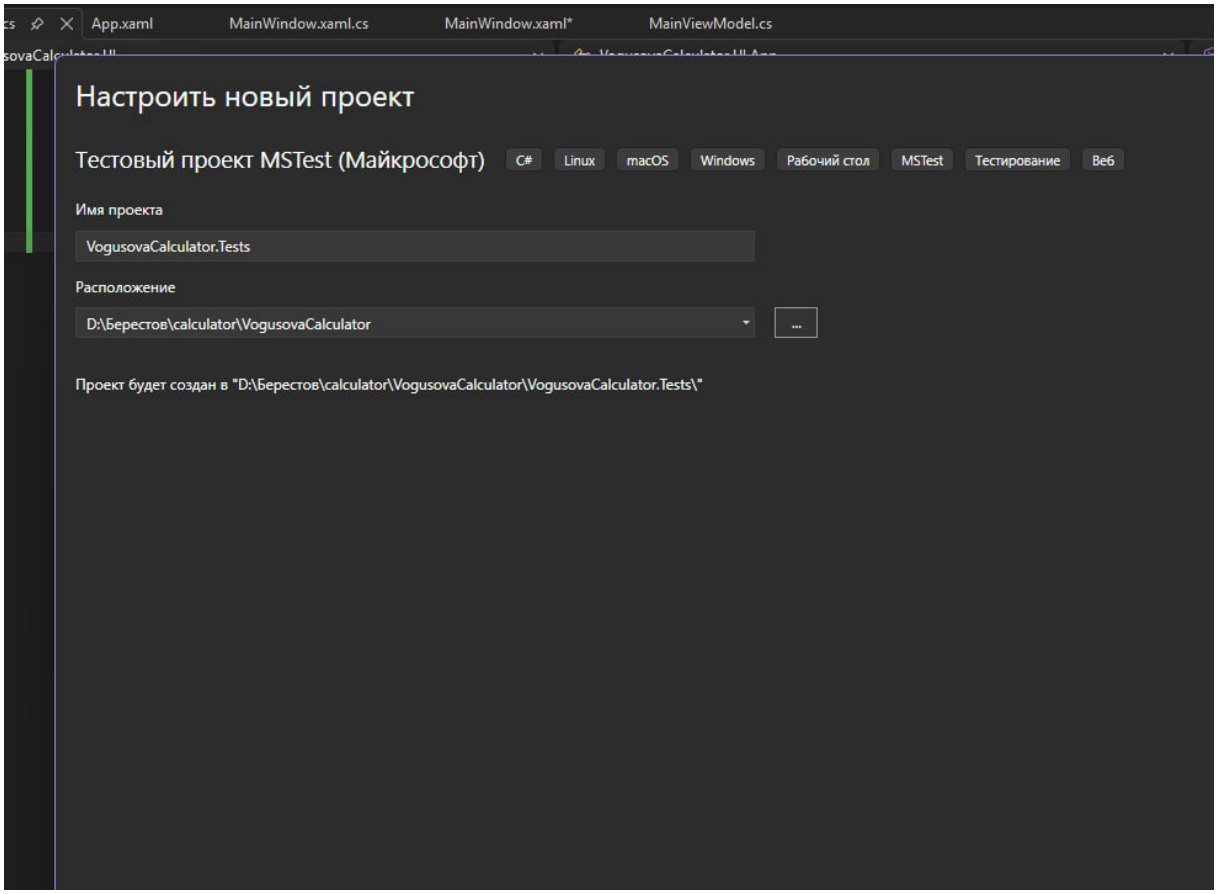
5.4 Подключение библиотеки логики

Для использования методов калькулятора в WPF-приложении была добавлена ссылка на проект VogusovaCalculator.Logic.

6. Разработка модульных тестов

6.1 Создание тестового проекта

Для проверки корректности работы библиотеки классов был создан проект модульных тестов VogusovaCalculator.Tests с использованием шаблона «Тестовый проект MSTest».



6.2 Написание тестов

Тесты покрывают все основные операции и обработку исключений:

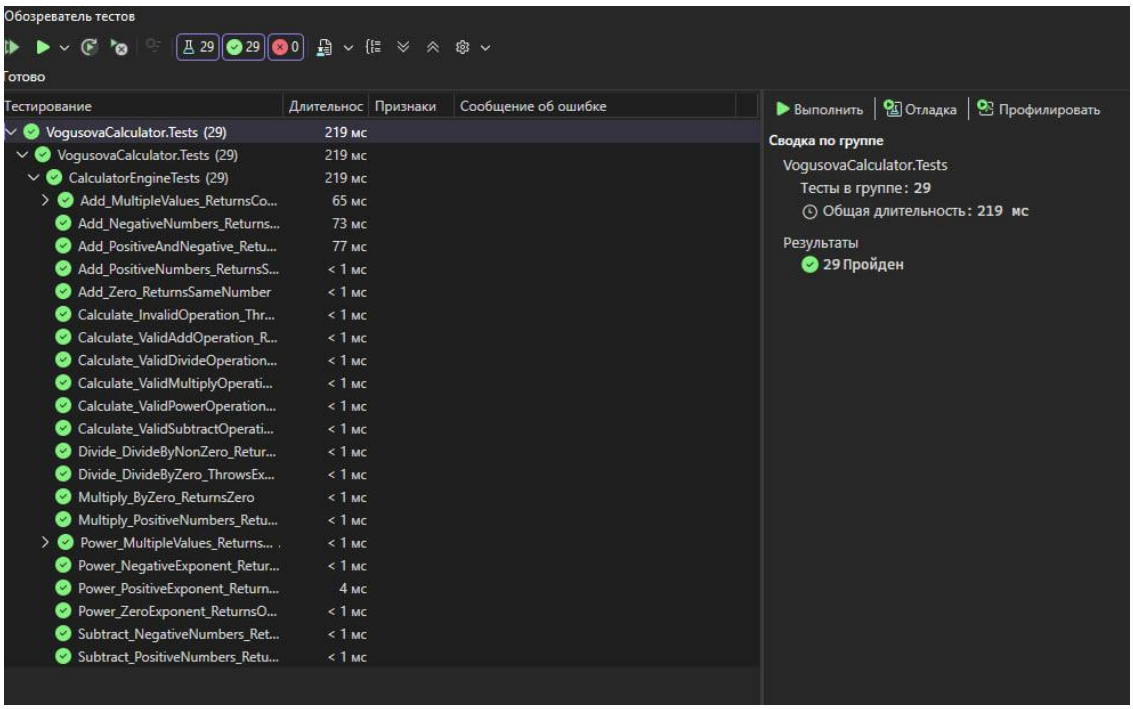
csharp

```
[TestClass]
public class CalculatorEngineTests
{
    private CalculatorEngine _engine = null!;

    [TestInitialize]
    public void Setup()
    {
        _engine = new CalculatorEngine();
    }

    [TestMethod]
    public void Add_PositiveNumbers_ReturnsSum()
    {
        double result = _engine.Add(5, 3);
        Assert.AreEqual(8, result, 0.001);
    }

    [TestMethod]
    public void Divide_DivideByZero_ThrowsException()
    {
        bool exceptionThrown = false;
        try
        {
            _engine.Divide(5, 0);
        }
```

```
        catch (DivideByZeroException)
        {
            exceptionThrown = true;
        }
        if (!exceptionThrown)
            Assert.Fail("Ожидалось исключение DivideByZeroException");
    }

    [TestMethod]
    [DataRow(5, 3, 8)]
    [DataRow(0, 0, 0)]
    [DataRow(-5, 3, -2)]
    public void Add_MultipleValues_ReturnsCorrectSum(double a, double b, double expected)
    {
        double result = _engine.Add(a, b);
        Assert.AreEqual(expected, result, 0.001);
    }
}
```

6.3 Запуск тестов

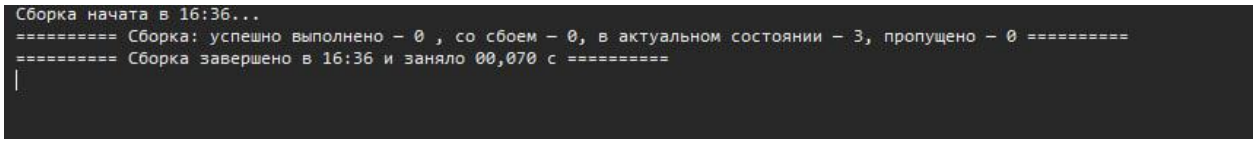
После написания тестов были запущены все тесты через меню «Тест» → «Запустить все тесты».



7. Сборка и тестирование

7.1 Сборка решения

Перед запуском приложения была выполнена сборка решения. Ошибок не обнаружено.



7.2 Проверка работы калькулятора

При запуске приложения была проверена работа всех операций:

Операция Ввод Результат

Сложение 5 + 3 = 8

Вычитание 10 - 4 = 6

Умножение 4 × 5 = 20

Деление 10 ÷ 2 = 5

Деление на ноль 5 ÷ 0 = Ошибка

Степень 2 ^ 3 = 8

8. Публикация на GitHub

8.1 Инициализация Git-репозитория

В Visual Studio была выполнена инициализация Git-репозитория с помощью встроенных средств.

8.2 Создание репозитория на GitHub

На сайте GitHub был создан публичный репозиторий VogusovaCalculator.

8.3 Отправка кода

Весь код проекта был зафиксирован и отправлен в удалённый репозиторий.

Ссылка на репозиторий:

https://github.com/irinavogysova-debug/VogusovaCalculator

9. Заключение

В ходе выполнения практической работы было создано приложение «Калькулятор», соответствующее всем требованиям задания:

1. Разработана библиотека классов VogusovaCalculator.Logic с методами для выполнения арифметических операций.

2. Создано WPF-приложение VogusovaCalculator.UI с удобным графическим интерфейсом, поддерживающим операции +, -, ×, ÷, ^, скобки и историю вычислений.

3. Реализован паттерн MVVM с использованием RelayCommand и MainViewModel, что обеспечивает разделение логики и представления.

4. Написаны модульные тесты с использованием MSTest, покрывающие все основные операции и обработку исключений.

5. Выполнена публикация проекта на GitHub.

Все тесты успешно проходят, приложение корректно обрабатывает ошибки (деление на ноль) и ведёт историю вычислений.

10. Список использованных источников

1. Microsoft Docs. WPF Documentation. – URL: https://learn.microsoft.com/ru-ru/dotnet/desktop/wpf/

2. Microsoft Docs. Unit Testing in Visual Studio. – URL: https://learn.microsoft.com/ru-ru/visualstudio/test/

3. Методические материалы по дисциплине «Поддержка и тестирование программных модулей».

4. GitHub Documentation. – URL: https://docs.github.com/ru